

InfoOS Agentic AI

Complete Technical Design, Multi-Agent Architecture, and Production Implementation Report

SYSTEM

InfoOS Desktop

STACK

Electron + React + Flask
+ SQLite

AGENT RUNNER

AgentGraph (Zero Pip
Dep)

SECURITY TIER

RBAC + Hardcoded
Ceilings

1. Executive Summary & Problem Space

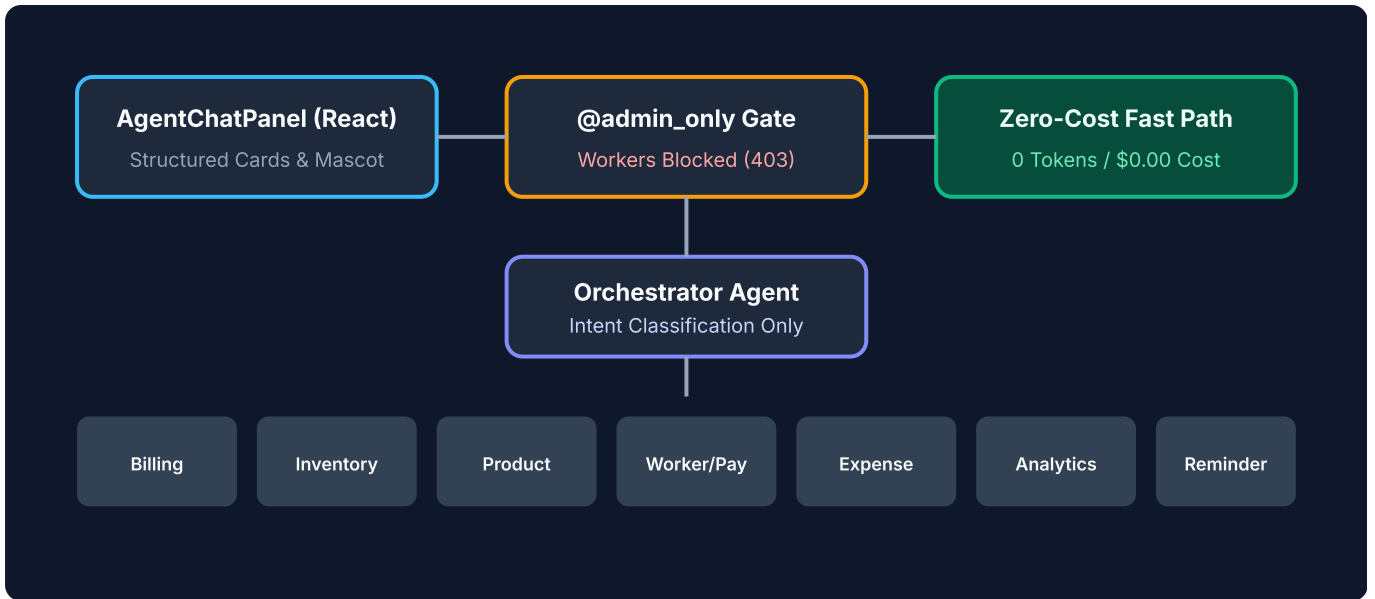
Retail Point-of-Sale (POS) and Enterprise Resource Planning (ERP) applications operate in zero-fault environments: inventory discrepancies cause stockouts, billing errors break legal tax compliance, and accidental data deletions permanently destroy transaction history.

Traditional AI integrations in business management software almost universally adopt one of two flawed paradigms:

- **Unbounded Text-to-SQL Bots:** The model generates raw SQLite/Postgres queries. This presents critical SQL injection risks, bypasses validation logic, and frequently causes database corruption.
- **Monolithic Single-Prompt Chatbots:** All tools are dumped into a single agent prompt, causing prompt degradation, high hallucination rates, and massive token costs.

The InfoOS Solution: InfoOS implements a guarded, multi-agent architecture with zero direct SQL generation. Every tool is a thin wrapper over existing, validated Python service layers. Financial and destructive actions are gated by human-in-the-loop diff cards, while high-frequency queries execute via a deterministic fast-path consuming **0 tokens and \$0.00 in LLM cost**.

High-Level System Topology



2. The Six Architectural Pillars

Pillar 1 Zero Raw SQL Guarantee

Agents possess zero capability to generate or execute raw SQL strings. Every tool maps to pre-existing, validated Python backend services (`DatabaseService` , `WorkerService`). Foreign key constraints, transaction rollbacks, and stock validations are automatically inherited.

Pillar 2 Zero-Cost Fast Path

High-frequency business inquiries (e.g., daily sales summaries, staff on duty, low-stock threshold queries) are intercepted via deterministic regex classifiers in `fast_path.py` . Handled in under 20ms with 0 input/output tokens and \$0.00 cost.

Pillar 3 Resumable State Graph

Implemented via `AgentGraph` with zero external pip dependencies. Checkpointed in SQLite (`AgentCheckpoint`). Destructive actions pause the execution state at `waiting_approval` and resume execution instantly when an admin approves.

Pillar 4 Hardcoded Security Ceilings

Worker accounts are hard-blocked at the middleware layer. Even for Admin users, actions involving payroll disbursement, retroactive bill voiding, or catalog deletion have hardcoded ceilings and can **never** be configured for full autonomy.

Pillar 5 Provider-Agnostic BYO-LLM

Businesses bring their own API keys for OpenAI, Anthropic Claude, Google Gemini, or local endpoints (Ollama/vLLM). All credentials are encrypted at rest with Fernet/AES-256 and never logged or exposed to the client.

Pillar 6 Structured UI Card Engine

Instead of unstructured markdown and emojis, the models stream strict JSON conforming to typed schemas (`metric_list` , `insight_block` , `action_list` , `table`). Directly rendered into native React cards with one-click PDF export.

3. Domain Agents & Scope Matrix

Domain Agent	Registered Capabilities	Default Tier	Safety & Policy Constraints
Billing Agent	Draft bills, lookup products, calculate split payments, hold tables	Suggest & Confirm	Hard Ceiling: Full autonomy locked. Cannot disburse funds or void old bills without human review.
Inventory Agent	Adjust stock levels, set low-stock alert thresholds, reorders	Suggest & Confirm	Stock changes require explicit audit reasons. In-memory cache cleared on mutation.
Product Agent	Add/edit items, variations, categories, item group sequences	Suggest & Confirm	Permanent deletion strictly forbidden. Soft-disables items (<code>active=False</code>) to protect sales history.
Worker / Payroll	Attendance audit, salary advances, active shift monitoring	Suggest & Confirm	Hard Ceiling: Full payroll disbursement requires explicit human manual execution.

Domain Agent	Registered Capabilities	Default Tier	Safety & Policy Constraints
Expense Agent	Log operational expenses, categorize vendor costs, report trends	Suggest & Confirm	Bulk deletion requires staged sample preview tables before confirmation.
Analytics Agent	Sales summaries, margin analysis, revenue comparisons	Full Autonomy	Read-only by construction. Completely safe; cannot alter database records.
Reminder Agent	Schedule operational alerts, vendor follow-ups, stock checks	Full Autonomy	Low risk, fully reversible operational reminders.

4. Resumable State Graph & SQLite Checkpointing

The state machine is built entirely with standard Python libraries and SQLAlchemy. When a tool requires human approval, the graph pauses, stages a proposal in `AgentActionLog`, and saves the conversation state to `AgentCheckpoint`.

```
# Graph Topology: backend/agents/graph_runner.py
call_llm → check_tool_calls → dispatch_tool → append_tool_result
          ↑
          |
(dispatch_tool pauses execution when status="waiting_approval")
(check_tool_calls terminates when status="done" or no pending tool calls)
```

The Suggest-and-Confirm Execution Flow

Human-in-the-Loop Diff Generation: When an admin asks *"Increase Cold Coffee price to ₹120"*, the Product Agent calls `propose_update_product`. The backend computes a diff string: `"Update product #14: Price → ₹120.00"` and returns an action ID. The frontend renders an **Approve / Reject** card. The database remains untouched until the admin explicitly clicks Approve.

5. Zero-Cost Fast Path & Token Economics

In POS software, up to 70% of assistant queries are repetitive status checks. Passing these through an external LLM adds 1–3 seconds of latency and costs \$0.005–\$0.02 per call.

Query Type	Deterministic Pattern	Latency	Tokens Used	Cost
Daily Revenue / Net Profit	<code>r"(today'?ss+sales revenue profit)"</code>	< 15 ms	0	\$0.00000
Staff Attendance Audit	<code>r"(whos+iss+present attendances+today)"</code>	< 18 ms	0	\$0.00000
Low Stock Critical Alert	<code>r"(checks+lows+stock lows+stocks+items)"</code>	< 20 ms	0	\$0.00000
Open-Ended Business Analysis	Full Orchestrator Routing → LLM Adapter	800–1800 ms	~850 tokens	~\$0.00045 (GPT-4o-mini)

6. Scoped Deletion & Undo Safety

To prevent accidental data loss, InfoOS distinguishes between **history-bearing entities** and **transient entities**:

- **History-Bearing (Products, Workers, Bills):** Hard deletes are physically blocked in code. Products are soft-disabled (`active=False`) so historical invoices and analytics remain valid.
- **Reversible Action Rollbacks:** The `UndoService` logs inverse JSON mutations. If a stock adjustment or item rename is approved, an "Undo" button remains active in the chat panel to restore previous state immediately.

7. LinkedIn Launch Package

Use the following pre-formatted copy and carousel storyboard to announce and showcase this architecture on LinkedIn.

Recommended Post Copy

Post Caption:

Most "AI integrations" in business software are just glorified chatbots connected to raw SQL queries.

In a real-world Point of Sale or ERP system, that's a recipe for disaster: hallucinated stock counts, runaway token bills, and zero auditability.

Over the past few months, we designed and built a production-grade Agentic AI layer into InfoOS (Electron + React + Flask + SQLite). Here are the 5 architectural rules we followed:

1

Zero Direct SQL: The agents never touch the database directly. All capabilities are thin wrappers over our existing, validated service functions.

2

Zero-Cost Fast Path: Common questions ("today's sales", "low stock audit", "staff on duty") bypass the LLM entirely via deterministic regex matching — 0 tokens, \$0.00 cost, and instant 15ms responses.

3

Resumable State Graph: Built a lightweight state machine with SQLite checkpoints. Destructive actions pause the execution state and resume when the admin clicks Approve.

4

Hardcoded Safety Ceilings: Cashiers/workers are blocked at the middleware layer (403). Payroll disbursement and bill voids can NEVER be automated.

5

Structured UI Cards: Replaced raw markdown dumps with strict JSON schemas rendered into native KPI cards, diff previews, and tables.

AI in enterprise software shouldn't just be a gimmick — it has to be reliable, transparent, and bounded by business logic.

#AgenticAI #SoftwareEngineering #Python #React #Electron #ArtificialIntelligence #SystemDesign #BuildingInPublic

8-Slide Carousel Storyboard (PDF Presentation)

Slide #	Headline	Key Visual / Content Elements
Slide 1	Building Production-Grade Agentic AI for Desktop POS	High-contrast cover slide with InfoOS logo & architectural badges.

Slide #	Headline	Key Visual / Content Elements
Slide 2	The Danger of Unbounded Text-to-SQL	Comparison: SQL Injection & Hallucinations vs Validated Service Layers.
Slide 3	Hub-and-Spoke Multi-Agent Architecture	Diagram showing Orchestrator routing to 7 domain-specific specialists.
Slide 4	Zero-Cost Fast Path: 0 Tokens, \$0.00 Cost	Regex classification table demonstrating instant <20ms performance.
Slide 5	Resumable State Graphs with SQLite Checkpointing	Flowchart of state execution pausing on <code>waiting_approval</code> .
Slide 6	Granular Permissions & Non-Negotiable Ceilings	Security matrix: Worker 403 blocks and mandatory human confirmation.
Slide 7	Structured UI Cards vs Markdown Text Dumps	Side-by-side: Messy emoji markdown vs Clean React KPI metric cards.
Slide 8	Summary & Key Engineering Takeaways	Key lessons, tech stack summary, and engagement call-to-action.